10 SOLID Design Principles

Lecture 12

Principle	Definition	Ask yourself	Patterns that support it
S — Single Responsibility	A class should have **only one reason to change**.	Can I describe the class without "and"?	Facade (coordinates; work in separate classes), Decorator (one responsibility per wrapper), State (one class per state), CoR (one check per handler)
O — Open/Closed	Open for **extension**, closed for **modification**.	Can I add behaviour without editing tested code?	Strategy, Decorator, Factory, Template Method, State, CoR, Adapter (new classes, no edits)
L — Liskov Substitution	Objects of a superclass must be **replaceable by subclass objects** without breaking correctness.	Can I safely replace the parent with the child?	Template Method, State, Proxy, Decorator all rely on substitutable subtypes
I — Interface Segregation	Clients should **not be forced to depend on methods they don't use**. Prefer small focused interfaces.	Am I forcing a class to implement methods it doesn't need?	Adapter (narrow Target), role interfaces (`Workable`, `Eatable`)
D — Dependency Inversion	High-level modules should not depend on low-level modules; **both depend on abstractions**. Details depend on abstractions.	Does business logic depend on a concrete class?	Factory (returns abstractions), Facade/Adapter/Proxy behind interfaces; Dependency Injection (Spring)

S — Single Responsibility

```
// BAD: Order changes for 4 reasons
public class Order {
    public double calculateTotal() { ... }   // business rules
    public void saveToDatabase()   { ... }   // persistence
    public void printInvoice()     { ... }   // presentation
    public void sendEmail()        { ... }   // communication
}
```

```
// GOOD: one responsibility each
public class Order          { double calculateTotal() {...} }
public class OrderRepository{ void save(Order o) {...} }
public class InvoicePrinter { void print(Order o) {...} }
public class EmailService   { void sendConfirmation(Order o)
{...} }
// OrderService (facade) coordinates them
```

"If the database technology changes, should `Order` change?" No ⇒ responsibilities separated. "This class calculates orders *and* saves orders *and* sends emails" ⇒ warning sign.

O — Open/Closed

```
// BAD: edit this method for every new customer type
public double calculateDiscount(String type, double amt) {
    if (type.equals("REGULAR")) return amt * 0.05;
    if (type.equals("PREMIUM")) return amt * 0.10;
    if (type.equals("VIP"))     return amt * 0.20;
    return 0;                 // + STUDENT? edit again
}
```

```
// GOOD: polymorphism (Strategy)
public interface DiscountStrategy { double calculate(double
amt); }
public class VipDiscount implements DiscountStrategy {
    public double calculate(double a) { return a * 0.20; } }
public class StudentDiscount implements DiscountStrategy {   //
NEW
    public double calculate(double a) { return a * 0.15; } }
public class DiscountCalculator {
    public double calculate(DiscountStrategy s, double a) {
return s.calculate(a); }
}
```

Same for payments: `PaymentMethod` interface; CryptoPayment added as a new class, `PaymentProcessor` unchanged.

L — Liskov Substitution

```
// VIOLATION 1: Square extends Rectangle
Rectangle r = new Square();
r.setWidth(5); r.setHeight(10);
r.getArea();          // expected 50, got 100

// VIOLATION 2: Penguin extends Bird
class Bird { void fly() {...} }
class Penguin extends Bird {
    void fly() { throw new UnsupportedOperationException(); }
}
makeItFly(new Penguin());   // crashes at runtime
```

```
// FIX: model the right abstraction
interface Shape { double getArea(); }
class Rectangle implements Shape { ... }
class Square    implements Shape { ... }

abstract class Bird { abstract void eat(); }
interface Flyable { void fly(); }
class Sparrow extends Bird implements Flyable { ... }
class Penguin extends Bird { ... }   // never asked to fly
```

A subtype must honour the parent's contract: same expectations, no stronger preconditions, no weaker postconditions, no surprising exceptions. Ask not only "is-a?" but "can the child fulfil the **behavioural contract** of the parent?"

I — Interface Segregation

```
// BAD: fat interface
interface Worker { void work(); void eat(); void sleep(); }
class Robot implements Worker {
    public void work() { ... }
    public void eat()   { throw new
UnsupportedOperationException(); }
    public void sleep() { throw new
UnsupportedOperationException(); }
}
```

```
// GOOD: small role interfaces (capabilities)
interface Workable  { void work(); }
interface Eatable   { void eat(); }
interface Sleepable { void sleep(); }
class Human implements Workable, Eatable, Sleepable { ... }
class Robot implements Workable { ... }
```

Java examples of capability interfaces: `Comparable`, `Runnable`, `Iterable`, `AutoCloseable`. Framework design: split `ApplicationComponent` into `Startable`, `Loggable`, `Cacheable`…

D — Dependency Inversion (+ Dependency Injection)

```
// BAD: high-level depends on low-level concrete class
public class UserService {
    private MySQLDatabase database = new MySQLDatabase();
    public void saveUser(String u) { database.save(u); }
}   // change DB -> change UserService; can't mock
```

```
// GOOD: depend on abstraction; inject via constructor
public interface UserRepository { void save(String user); }
public class MySQLUserRepository implements UserRepository { ...
}
public class PostgreSQLUserRepository implements UserRepository
{ ... }
public class UserService {
    private final UserRepository repository;
    public UserService(UserRepository repository) {
this.repository = repository; }
    public void saveUser(String u) { repository.save(u); }
}
new UserService(new PostgreSQLUserRepository());   // or a
FakeRepository in tests
```

Dependency Inversion = the principle (depend on abstractions). **Dependency Injection** = the technique (dependencies are provided from outside, e.g. constructor injection; Spring container wires them). Benefits: loose coupling, easy to switch implementations, testable with fakes/mocks.

Exam use & key points

SOLID vs patterns	SOLID = principles (“how should I design classes?”); patterns = proven solutions to recurring problems; frameworks = reusable infrastructure (Spring applies DIP/DI). A pattern does not automatically guarantee SOLID — it depends on how you apply it.
Not “more classes”	SOLID is about managing **change**, not creating interfaces for everything (over-engineering a 2-line calculator violates YAGNI). Add abstraction when change is likely.
Benefits	Smaller ripple effect of change · easier testing (DIP + mocks) · maintainability · extensibility · supports clean architecture (dependencies point inward to business logic).
Linking in answers	When justifying a pattern, add one SOLID line: Decorator/Strategy/Factory ⇒ OCP; Facade/CoR handlers ⇒ SRP; State/Proxy/Template subclasses ⇒ LSP; Adapter Target ⇒ ISP; Factory/DI/Singleton-behind-interface ⇒ DIP. “Explain the use of interfaces” ⇒ contract + polymorphism + DIP + OCP + testability.